LAPORAN PRAKTIKUM
KECERDASAN BUATAN



RUBEN SIBARANI
NIM: 42324037

INSTITUT TEKNOLOGI DEL
D3 TEKNOLOGI INFORMASI

```python
import pandas as pd
df = pd.read_csv('Text_classification.csv')
print(df.head())
```

[2] Python

```
  Category                                            Message
0      ham  Go until jurong point, crazy.. Available only ...
1      ham                      Ok lar... Joking wif u oni...
2     spam  Free entry in 2 a wkly comp to win FA Cup fina...
3      ham  U dun say so early hor... U c already then say...
4      ham  Nah I don't think he goes to usf, he lives aro...
```

Code diatas untuk menggunakan pandas sebagai library

dan menggunakan dataset Text_classification.csv

```python
# Download resource NLTK yang diperlukan
import nltk
nltk.download('punkt_tab')
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
# Import library untuk preprocessing
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
# Inisialisasi lemmatizer
lemmatizer = WordNetLemmatizer()
# Fungsi preprocessing teks
def clean_text(text):
    text = text.lower() # Ubah menjadi lowercase
    text = re.sub(r'\d+', '', text) # Hapus angka
    text = text.translate(str.maketrans('', '', string.punctuation)) # Hapus tanda baca
    words = word_tokenize(text) # Tokenisasi
    words = [word for word in words if word not in stopwords.words('english')] # Hapus stopwords
    words = [lemmatizer.lemmatize(word) for word in words] # Lemmatization
    return ' '.join(words)
# Terapkan preprocessing ke teks
df['clean_text'] = df['Message'].apply(clean_text)
# Cek hasil preprocessing
print("\nHasil Preprocessing:\n", df[['Message', 'clean_text']].head())
```

[3] Python

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Ruben\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Ruben\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt.zip.
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Ruben\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\Ruben\AppData\Roaming\nltk_data...

Hasil Preprocessing:
                                                Message  \
0  Go until jurong point, crazy.. Available only ...
1                      Ok lar... Joking wif u oni...
2  Free entry in 2 a wkly comp to win FA Cup fina...
3  U dun say so early hor... U c already then say...
4  Nah I don't think he goes to usf, he lives aro...

                                          clean_text
0  go jurong point crazy available bugis n great ...
1                            ok lar joking wif u oni
2  free entry wkly comp win fa cup final tkts st ...
3                u dun say early hor u c already say
4           nah dont think go usf life around though
```

import nltk
nltk.download('punkt_tab')
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
Bagian ini mengunduh sumber daya dari pustaka nltk yang diperlukan untuk tahapan pemrosesan teks berikutnya, seperti tokenisasi, daftar kata-kata umum, dan basis data leksikal WordNet.
==========

import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

re: Modul untuk ekspresi reguler (regular expression) guna mencocokkan dan membersihkan pola karakter tertentu.

string: Menyediakan konstanta string, termasuk kumpulan tanda baca (punctuation).

stopwords: Kumpulan kata umum yang biasanya tidak memiliki makna analitik penting

word_tokenize: Fungsi untuk memecah kalimat menjadi daftar kata-kata (token).

df['clean_text'] = df['Message'].apply(clean_text)

print("\nHasil Preprocessing:\n", df[['Message', 'clean_text']].head())

Kode ini menerapkan fungsi clean_text ke setiap baris data pada kolom 'Message' di dalam sebuah Pandas DataFrame (df), lalu hasilnya disimpan ke kolom baru bernama 'clean_text'.

Baris terakhir mencetak 5 baris pertama (.head()) dari kolom pesan asli ('Message') berdampingan dengan hasil pembersihannya ('clean_text') untuk memverifikasi apakah proses berhasil dengan baik.

```python
# Ubah label dari 'ham'/'spam' menjadi 0/1
df['Category'] = df['Category'].map({'ham': 0, 'spam': 1})
# Cek jumlah masing-masing kelas
print(df['Category'].value_counts())
```

Python

```
Category
0    4825
1     747
Name: count, dtype: int64
```

df['Category'] = df['Category'].map({'ham': 0, 'spam': 1})

Metode .map({...}) digunakan untuk menerjemahkan nilai string pada kolom 'Category':
'ham' (pesan normal/bukan spam) diubah menjadi 0.
'spam' (pesan sampah/penipuan) diubah menjadi 1

```python
from sklearn.model_selection import train_test_split
# Pisahkan fitur dan label
X = df['clean_text']
y = df['Category']
# Split data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Python

X = df['clean_text']: Menetapkan kolom teks yang sudah dibersihkan (clean text) sebagai fitur atau variabel independen yang akan dipelajari oleh model.

y = df['Category']: Menetapkan kolom kategori (0 atau 1) sebagai label atau target yang ingin diprediksi oleh model.

test_size=0.2: Menentukan proporsi pembagian data, di mana 20% dari total data dialokasikan sebagai data uji (X_test, y_test) dan 80% sisanya dialokasikan sebagai data latih.

random_state=42: Berfungsi sebagai angka acak awal agar hasil pengacakan pembagian data selalu konsisten dan bisa direproduksi setiap kali kode dijalankan kembali.

=y: Memastikan bahwa proporsi kelas (0 dan 1) pada data latih dan data uji memiliki perbandingan yang sama persis dengan data aslinya. Hal ini sangat penting untuk mencegah bias terutama jika dataset Anda tidak seimbang.

```
from sklearn.feature_extraction.text import TfidfVectorizer
# Konversi teks menjadi fitur numerik menggunakan TF-IDF
vectorizer = TfidfVectorizer(ngram_range=(1,2), max_features=5000) # Menggunakan bigram
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
print("\nTF-IDF Shape (Train):", X_train_tfidf.shape)
```

[6] Python

```
TF-IDF Shape (Train): (4457, 5000)
```

TfidfVectorizer: Mengubah kumpulan dokumen teks menjadi matriks nilai TF-IDF. TF-IDF mengukur seberapa penting suatu kata dalam dokumen relatif terhadap keseluruhan korpus (kumpulan teks).

ngram_range=(1,2): Menginstruksikan model untuk mengambil kata tunggal (unigram, misal: "not", "good") sekaligus kombinasi dua kata berurutan (bigram, misal: "not good"). Hal ini membantu model menangkap konteks frasa atau negasi.

max_features=5000: Membatasi jumlah fitur (kosakata/vocabulary) maksimal sebanyak 5.000 kata atau frasa teratas yang memiliki skor TF-IDF paling penting secara global, guna mengurangi kompleksitas komputasi.

```
from sklearn.naive_bayes import MultinomialNB
# Inisialisasi model Naïve Bayes
model = MultinomialNB(alpha=0.1)
# Train model
model.fit(X_train_tfidf, y_train)
# Prediksi hasil
y_pred = model.predict(X_test_tfidf)
```

from sklearn.naive_bayes import MultinomialNB

Mengimpor kelas MultinomialNB dari pustaka scikit-learn. Algoritma ini sangat populer dan efektif untuk tugas-tugas klasifikasi teks (seperti deteksi spam) karena bekerja dengan baik pada data fitur diskrit yang merepresentasikan frekuensi kemunculan kata (seperti matriks TF-IDF).

model = MultinomialNB(alpha=0.1)

Membuat objek model *Multinomial Naive Bayes*.

Parameter alpha=0.1 adalah parameter penghalusan (*Laplace/Lidstone smoothing*). Nilai ini berfungsi untuk mencegah pembagian dengan nilai nol apabila model menemukan kata baru pada data uji yang tidak pernah muncul sama sekali di dalam data latih.

Metode .fit() melatih model menggunakan data latih. Model mempelajari pola statistik hubungan antara fitur teks berbobot TF-IDF (X_train_tfidf) dengan label kelas targetnya masing-masing (y_train, yaitu 0 untuk *ham* dan 1 untuk *spam*).

```python
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Akurasi
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)
# Classification Report
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Python

accuracy_score: Untuk menghitung tingkat ketepatan global model.

classification_report: Untuk menampilkan ringkasan metrik performa lengkap per kelas (Precision, Recall, F1-Score, dan Support).

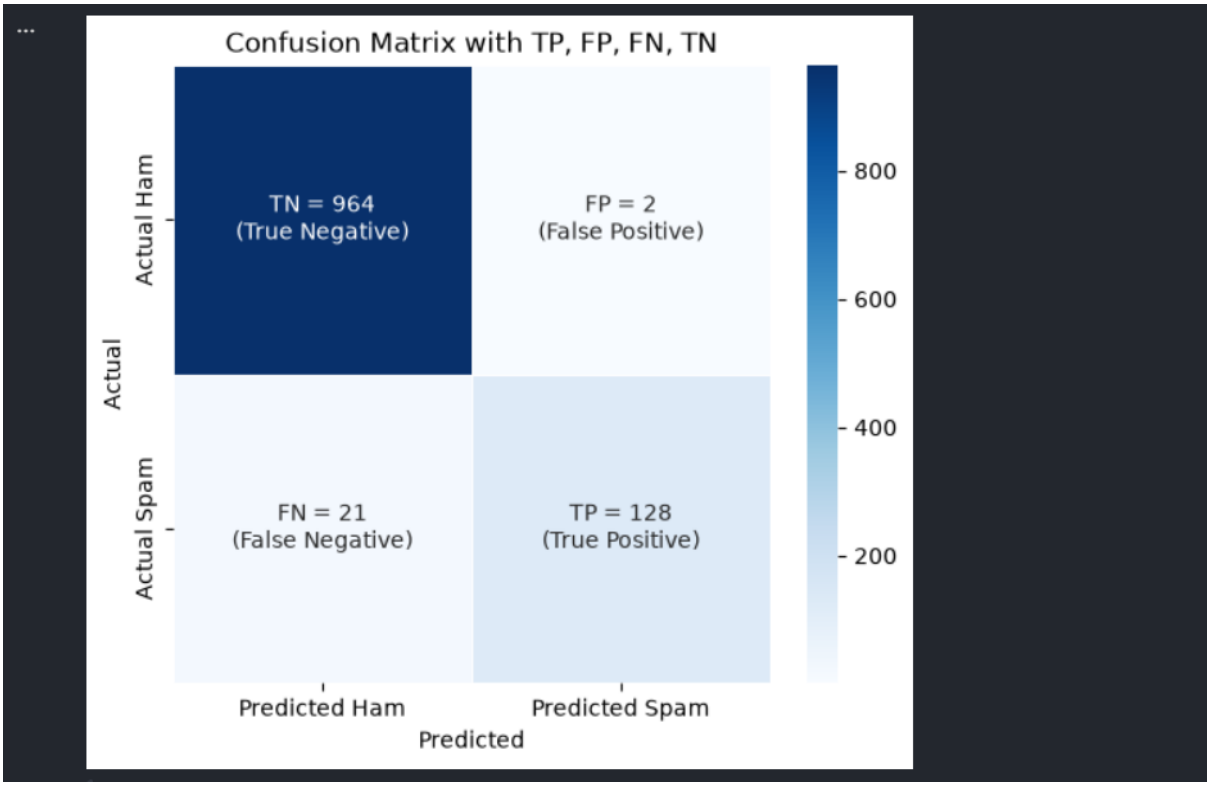
confusion_matrix: Untuk melihat detail kesalahan prediksi per kelas (meskipun pada baris ini diimpor, perhitungannya sudah digunakan pada blok kode visualisasi sebelumnya).

accuracy_score(y_test, y_pred) membandingkan label asli (y_test) dengan hasil tebakan model (y_pred), lalu menghitung persentase total prediksi yang benar dari keseluruhan data uji.

Nilai akurasi yang mendekati 1.0 (atau 100%) menunjukkan performa model yang semakin baik dalam mengklasifikasikan pesan spam maupun ham secara keseluruhan

```python
# Confusion Matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix
# Hitung confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
# Definisi elemen dalam confusion matrix
TN, FP, FN, TP = conf_matrix.ravel()
# Buat dataframe confusion matrix
df_conf_matrix = pd.DataFrame(conf_matrix, index=['Actual Ham', 'Actual Spam'], columns=['Predicted Ha
# Label untuk TP, FP, FN, TN
labels = np.array([["TN = {}\n(True Negative)".format(TN), "FP = {}\n(False Positive)".format(FP)],
 ["FN = {}\n(False Negative)".format(FN), "TP = {}\n(True Positive)".format(TP)]])
 # Plot heatmap dengan keterangan
plt.figure(figsize=(6,5))
ax = sns.heatmap(df_conf_matrix, annot=labels, fmt='', cmap='Blues', linewidths=0.5, cbar=True)
# Tambahkan label
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix with TP, FP, FN, TN')
plt.show()
```

Python




```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix
```

Penjelasan: Mengimpor modul yang diperlukan untuk perhitungan statistik evaluasi (confusion_matrix) dan visualisasi grafik (seaborn, matplotlib, numpy, pandas).

Confusion_matrix(y_test, y_pred): Membandingkan label asli (y_test) dengan hasil prediksi model (y_pred) untuk menghasilkan matriks performa.

==========

plt.figure(figsize=(6,5)): Mengatur ukuran jendela gambar grafik.

sns.heatmap(...): Menggambar grafik matriks dalam bentuk peta panas (heatmap):

annot=labels: Mengisi kotak-kotak grafik dengan teks label kustom yang sudah dibuat sebelumnya.

fmt='': Mengosongkan format bawaan karena teks anotasi sudah berbentuk string kustom.

cmap='Blues': Menggunakan gradasi warna biru untuk menunjukkan tingkat kepadatan/jumlah nilai.

linewidths=0.5: Memberikan garis pemisah antar-kotak agar terlihat rapi.

plt.xlabel, plt.ylabel, plt.title: Menambahkan keterangan label pada sumbu X, sumbu Y, serta judul grafik.

plt.show(): Perintah untuk merender dan menampilkan grafik ke layar.